目录

致所有读者
致教师
致学生
致谢
写在最后
参考文献

1 关于本书的对话

2 操作系统简介
2.1 虚拟化 CPU
2.2 虚拟化内存
2.3 并发
2.4 持久性
2.5 设计目标
2.6 一点历史
2.7 小结
参考文献
作业

第一部分 虚拟化

3 关于虚拟化的对话

4 抽象：进程
4.1 抽象：进程
4.2 进程 API
4.3 创建进程：再多讲一点
4.4 进程状态
4.5 数据结构
4.6 小结
参考文献
作业（模拟）

5 插叙：进程 API
5.1 `fork()` 系统调用

5.2 wait() 系统调用
5.3 最后是 exec() 系统调用
5.4 为什么？API 的设计动机
5.5 进程控制与用户
5.6 实用工具
5.7 小结
参考文献
作业（模拟）
作业（编程）

6 机制：受限直接执行
6.1 基本技术：受限直接执行
6.2 问题 1：受限的操作
6.3 问题 2：在进程之间切换
6.4 担心并发?
6.5 小结
参考文献
作业（测量）

7 调度：简介
7.1 工作负载假设
7.2 调度指标
7.3 先进先出（FIFO）
7.4 最短作业优先（SJF）
7.5 最短完成时间优先（STCF）
7.6 新的指标：响应时间
7.7 时间片轮转
7.8 结合 I/O
7.9 不再有先知
7.10 小结
参考文献
作业（模拟）

8 调度：多级反馈队列
8.1 MLFQ：基本规则
8.2 尝试 1：如何改变优先级
8.3 尝试 2：优先级提升
8.4 尝试 3：更好的记账
8.5 MLFQ 的调优及其他问题
8.6 MLFQ：小结
参考文献
作业（模拟）

9 调度：比例份额
9.1 基本概念：彩票代表你的份额
9.2 彩票机制
9.3 实现

9.4 一个例子
9.5 怎样分配彩票?
9.6 步长调度
9.7 Linux 的完全公平调度程序（CFS）
9.8 小结
参考文献
作业（模拟）

10 多处理器调度（进阶）
10.1 背景：多处理器架构
10.2 别忘了同步
10.3 最后一个问题：缓存亲和性
10.4 单队列调度
10.5 多队列调度
10.6 Linux 多处理器调度程序
10.7 小结
参考文献
作业（模拟）

11 关于 CPU 虚拟化的总结对话

12 关于内存虚拟化的对话

13 抽象：地址空间
13.1 早期系统
13.2 多道程序设计和分时
13.3 地址空间
13.4 目标
13.5 小结
参考文献
作业（编程）

14 插叙：内存 API
14.1 内存类型
14.2 `malloc()` 调用
14.3 `free()` 调用
14.4 常见错误
14.5 底层操作系统支持
14.6 其他调用
14.7 小结
参考文献
作业（编程）

15 机制：地址转换
15.1 假设
15.2 一个例子
15.3 动态（基于硬件的）重定位
15.4 硬件支持：总结

15.5 操作系统层面的问题
15.6 小结
参考文献
作业（模拟）

16 分段
16.1 分段：泛化的基址/界限
16.2 引用的是哪个段?
16.3 栈怎么办?
16.4 支持共享
16.5 细粒度与粗粒度的分段
16.6 操作系统支持
16.7 小结
参考文献
作业（模拟）

17 空闲空间管理
17.1 假设
17.2 底层机制
17.3 基本策略
17.4 其他方法
17.5 小结
参考文献
作业（模拟）

18 分页：简介
18.1 一个简单的例子和概览
18.2 页表存放在哪里?
18.3 页表里究竟有什么?
18.4 分页：不仅太大，而且太慢
18.5 一次内存跟踪
18.6 小结
参考文献
作业（模拟）

19 分页：更快的转换（TLB）
19.1 TLB 的基本算法
19.2 示例：访问数组
19.3 谁来处理 TLB 未命中?
19.4 TLB 的内容：里面有什么?
19.5 TLB 的问题：上下文切换
19.6 问题：替换策略
19.7 一个真实的 TLB 项
19.8 小结
参考文献
作业（测量）

 © 2008–23, ARPACI-DUSSEAU

20 分页：更小的页表
20.1 简单的办法：更大的页
20.2 混合方案：分页和分段
20.3 多级页表
20.4 反置页表
20.5 把页表交换到磁盘
20.6 小结
参考文献
作业（模拟）

21 超越物理内存：机制
21.1 交换空间
21.2 存在位
21.3 缺页异常
21.4 内存满了怎么办?
21.5 缺页异常的控制流
21.6 置换实际上何时发生
21.7 小结
参考文献
作业（测量）

22 超越物理内存：策略
22.1 缓存管理
22.2 最优置换策略
22.3 一个简单的策略：FIFO
22.4 另一个简单的策略：随机
22.5 利用历史：LRU
22.6 工作负载示例
22.7 实现基于历史的算法
22.8 近似 LRU
22.9 考虑脏页
22.10 其他虚拟内存策略
22.11 抖动
22.12 小结
参考文献
作业（模拟）

23 完整的虚拟内存系统
23.1 VAX/VMS 虚拟内存
23.2 Linux 虚拟内存系统
23.3 小结
参考文献

24 关于内存虚拟化的总结对话

第二部分 并发

25 关于并发的对话

26 并发：简介
26.1 为什么要用线程?
26.2 一个例子：创建线程
26.3 为什么会更糟：共享数据
26.4 问题的核心：不受控制的调度
26.5 对原子性的期望
26.6 另一个问题：等待另一个线程
26.7 小结：为什么要在操作系统课上讲这些?
参考文献
作业（模拟）

27 插叙：线程 API
27.1 创建线程
27.2 等待线程结束
27.3 锁
27.4 条件变量
27.5 编译和运行
27.6 小结
参考文献
作业（编程）

28 锁
28.1 锁的基本思想
28.2 Pthread 锁
28.3 实现一个锁
28.4 评价锁
28.5 控制中断
28.6 失败的尝试：只用加载和存储
28.7 用测试并设置实现可用的自旋锁
28.8 评价自旋锁
28.9 比较并交换
28.10 链接加载和条件存储
28.11 获取并增加
28.12 自旋太多：怎么办?
28.13 简单的办法：让出来就行，宝贝
28.14 使用队列：用睡眠代替自旋
28.15 不同的操作系统，不同的支持
28.16 两阶段锁
28.17 小结
参考文献
作业（模拟）

 © 2008–23, ARPACI-DUSSEAU

29 基于锁的并发数据结构
29.1 并发计数器
29.2 并发链表
29.3 并发队列
29.4 并发散列表
29.5 小结
参考文献
作业（编程）

30 条件变量
30.1 定义和例程
30.2 生产者/消费者（有界缓冲区）问题
30.3 覆盖条件
30.4 小结
参考文献
作业（编程）

31 信号量
31.1 信号量的定义
31.2 二值信号量（锁）
31.3 用信号量控制顺序
31.4 生产者/消费者（有界缓冲区）问题
31.5 读写锁
31.6 哲学家就餐问题
31.7 线程节流
31.8 如何实现信号量
31.9 小结
参考文献
作业（编程）

32 常见并发问题
32.1 有哪些类型的 bug?
32.2 非死锁 bug
32.3 死锁 bug
32.4 小结
参考文献
作业（编程）

33 基于事件的并发（进阶）
33.1 基本思想：事件循环
33.2 一个重要的 API：`select()`（或 `poll()`）
33.3 使用 `select()`
33.4 为什么更简单？不需要锁
33.5 一个问题：阻塞的系统调用
33.6 一个解决方案：异步 I/O
33.7 另一个问题：状态管理
33.8 使用事件时仍然存在的难题

 © 2008–23, ARPACI-DUSSEAU

33.9 小结
参考文献
作业（编程）

34 关于并发的总结对话

第三部分 持久性

35 关于持久性的对话

36 I/O 设备
36.1 系统架构
36.2 典型设备
36.3 典型协议
36.4 用中断降低 CPU 开销
36.5 用 DMA 更高效地搬运数据
36.6 设备交互的方法
36.7 融入操作系统：设备驱动程序
36.8 案例研究：一个简单的 IDE 磁盘驱动程序
36.9 历史注记
36.10 小结
参考文献

37 硬盘驱动器
37.1 接口
37.2 基本几何结构
37.3 一个简单的磁盘驱动器
37.4 I/O 时间：算一算
37.5 磁盘调度
37.6 小结
参考文献
作业（模拟）

38 廉价磁盘冗余阵列（RAID）
38.1 接口与 RAID 内部结构
38.2 故障模型
38.3 如何评估 RAID
38.4 RAID 0 级：条带化
38.5 RAID 1 级：镜像
38.6 RAID 4 级：用奇偶校验节省空间
38.7 RAID 5 级：轮转奇偶校验
38.8 RAID 比较：小结
38.9 RAID 的其他有趣问题
38.10 小结
参考文献
作业（模拟）

39 插叙：文件和目录
39.1 文件和目录
39.2 文件系统接口
39.3 创建文件
39.4 读写文件
39.5 读写，但不按顺序
39.6 共享文件表条目：`fork()` 和 `dup()`
39.7 用 `fsync()` 立即写入
39.8 重命名文件
39.9 获取文件信息
39.10 删除文件
39.11 创建目录
39.12 读取目录
39.13 删除目录
39.14 硬链接
39.15 符号链接
39.16 权限位和访问控制列表
39.17 创建和挂载文件系统
39.18 小结
参考文献
作业（编程）

40 文件系统实现
40.1 思考的方式
40.2 整体组织
40.3 文件组织：inode
40.4 目录组织
40.5 空闲空间管理
40.6 访问路径：读和写
40.7 缓存和缓冲
40.8 小结
参考文献
作业（模拟）

41 局部性和快速文件系统
41.1 问题：性能很差
41.2 FFS：解决方案是了解磁盘
41.3 组织结构：柱面组
41.4 策略：如何分配文件和目录
41.5 测量文件的局部性
41.6 大文件的例外规则
41.7 关于 FFS 的其他几件事
41.8 小结
参考文献
作业（模拟）

 © 2008–23, ARPACI-DUSSEAU

42 崩溃一致性：FSCK 和日志
42.1 一个详细的例子
42.2 方案一：文件系统检查程序
42.3 方案二：日志（或预写日志）
42.4 方案三：其他方法
42.5 小结
参考文献
作业（模拟）

43 日志结构文件系统
43.1 顺序地写入磁盘
43.2 顺序而高效地写入
43.3 要缓冲多少?
43.4 问题：找到 inode
43.5 用间接层来解决：inode map
43.6 完成方案：检查点区域
43.7 从磁盘读取文件：回顾
43.8 目录怎么办?
43.9 新的问题：垃圾回收
43.10 判断块是否存活
43.11 策略问题：清理哪些块，何时清理?
43.12 崩溃恢复和日志
43.13 小结
参考文献
作业（模拟）

44 基于闪存的 SSD
44.1 存储单个位
44.2 从位到 bank/平面
44.3 闪存的基本操作
44.4 闪存的性能和可靠性
44.5 从原始闪存到基于闪存的 SSD
44.6 FTL 的组织方式：一种糟糕的方法
44.7 日志结构的 FTL
44.8 垃圾回收
44.9 映射表的大小
44.10 磨损均衡
44.11 SSD 的性能和成本
44.12 小结
参考文献
作业（模拟）

45 数据完整性与保护
45.1 磁盘故障模式
45.2 处理 latent sector error
45.3 检测损坏：校验和
45.4 使用校验和

45.5 一个新问题：misdirected write
45.6 最后一个问题：lost write
45.7 Scrubbing
45.8 校验和的开销
45.9 小结
参考文献
作业（模拟）
作业（编程）

46 关于持久性的总结对话

47 关于分布式的对话

48 分布式系统
48.1 通信基础
48.2 不可靠的通信层
48.3 可靠的通信层
48.4 通信抽象
48.5 远程过程调用（RPC）
48.6 小结
参考文献
作业（编程）

49 Sun 的网络文件系统（NFS）
49.1 一个基本的分布式文件系统
49.2 走近 NFS
49.3 重点：简单、快速的服务器崩溃恢复
49.4 快速崩溃恢复的关键：无状态
49.5 NFSv2 协议
49.6 从协议到分布式文件系统
49.7 用幂等操作处理服务器故障
49.8 提高性能：客户端缓存
49.9 缓存一致性问题
49.10 评估 NFS 的缓存一致性
49.11 对服务器端写缓冲的影响
49.12 小结
参考文献
作业（测量）

50 Andrew 文件系统（AFS）
50.1 AFS 第 1 版
50.2 第 1 版的问题
50.3 改进协议
50.4 AFS 第 2 版
50.5 缓存一致性
50.6 崩溃恢复
50.7 AFSv2 的规模与性能
50.8 AFS 的其他改进

 © 2008–23, ARPACI-DUSSEAU

50.9 小结
参考文献
作业（模拟）

51 关于分布式的总结对话

 © 2008–23, ARPACI-DUSSEAU